

TruthDB capability catalogue

Extensive feature inventory inspired by Kafka, Elasticsearch, event sourcing, Redux, SQL Server, PostgreSQL, and adjacent data platforms

Prepared as a product-design requirements document for the TruthDB vision

Document purpose	Catalogue the major capabilities TruthDB should harvest from modern stream, search, event-sourced, relational, and client-state systems.
Document posture	Ambitious and exhaustive. This is deliberately broader than an MVP and is intended to inform long-term product architecture and staged roadmap decisions.
Structure	62 feature definitions grouped into 9 capability domains, each with a short rationale and concrete acceptance criteria.
Use	Reference for architecture, roadmap decomposition, design reviews, and acceptance-based delivery planning.

How to read this document

This is a requirements catalogue, not a release plan. A feature appearing here means it is strategically desirable, not that it belongs in a first milestone. The goal is to harvest the strongest ideas from existing systems and describe them in a way that is implementable, reviewable, and testable.

Design interpretation rules

- Feature descriptions explain intent and the kind of user or operator problem the feature should solve.
- Acceptance criteria are written at a product level. They should later be refined into protocol, storage-engine, and UI-specific tests.
- Some features overlap by design. In a hybrid system, the same business outcome often requires storage, query, security, and operational support together.
- A future roadmap should tag each feature as core platform, optional module, enterprise layer, or long-horizon differentiator.

Source inspirations

The feature set is synthesized primarily from official product documentation for Apache Kafka, Elasticsearch, PostgreSQL, SQL Server / Azure SQL, Redux, and Azure Architecture Center guidance for event sourcing and CQRS.

Capability domains

Domain	Primary inspirations	Feature IDs	Intent
Log, storage, and durability	Kafka, event stores, database recovery	F-001 to F-008	Durable append, ordering, replay, recovery, and retention.
Event sourcing and domain history	Event sourcing, CQRS	F-009 to F-016	Aggregates, projections, corrections, and schema evolution.
Relational database capabilities	PostgreSQL, SQL Server	F-017 to F-024	Constraints, MVCC, indexes, temporal state, and views.
Search and analytics	Elasticsearch	F-025 to F-032	Shards, mappings, full-text retrieval, aggregations, lifecycle.
Replication and integration	Kafka, PostgreSQL, SQL Server	F-033 to F-038	CDC, logical replication, connectors, failover, backfills.
Developer ergonomics	Redux, SQL, reactive	F-039 to F-046	Unified API, selectors,

and APIs	systems		subscriptions, cursoring, tooling.
Security and governance	Enterprise databases	F-047 to F-052	Identity, RLS, tenant isolation, audit, retention governance.
Operations and performance	Kafka, PostgreSQL, Elasticsearch	F-053 to F-058	Observability, workload classes, maintenance, backups, upgrades.
Advanced differentiators	Hybrid design opportunity	F-059 to F-062	Cross-model lineage, recomputation, explicit consistency modes.

1. Log, storage, and durability core

These features define the physical and logical truth model: how data is appended, ordered, retained, replayed, compacted, versioned, and recovered.

F-001 Append-only immutable commit log

TruthDB should treat the event log as the primary source of truth. Writes append new facts rather than mutating prior facts in place. This mirrors event stores and Kafka-style logs and provides a durable foundation for replay, audit, branching, and downstream materialization.

Acceptance criteria

- Given an accepted write, the system persists it as a new immutable record with a durable offset or sequence number.
- Existing committed records cannot be updated in place through normal APIs.
- Delete semantics are represented as tombstones or domain events, not silent physical rewrites.
- All reads of raw history can reconstruct the exact ordered stream for a partition, entity, or tenant.
- Crash recovery preserves prefix integrity: after restart, every visible record is fully committed and ordered.

F-002 Ordered streams with explicit partitioning

TruthDB should support ordered streams that preserve order within a partition while allowing horizontal scale across many partitions. This is essential for Kafka-like throughput and for per-aggregate event sourcing consistency.

Acceptance criteria

- A write can be routed by partition key, aggregate key, tenant key, or system strategy.
- Order is guaranteed within a single partition and explicitly not assumed across partitions unless a higher-level coordinator is used.
- Consumers can resume from a stable offset or cursor within each partition.
- Partition assignment metadata is queryable through admin and client APIs.
- Repartitioning operations expose operational state, progress, and any temporary constraints.

F-003 Idempotent write path

TruthDB should prevent duplicate persistence caused by retries, leader failover, network ambiguity, or client reconnects. The feature should resemble Kafka idempotent producer behavior and API-level idempotency keys common in durable systems.

Acceptance criteria

- A client can submit an idempotency key or producer sequence context with a write.
- Retrying the same logical write does not create duplicate committed records.
- The system returns whether the operation was newly committed or recognized as a duplicate.

- Idempotency windows, expiration, and storage costs are configurable per workload class.
- Duplicate suppression remains correct across restart and leader handover events.

F-004 Atomic multi-record transactions

TruthDB should support grouping several writes into one atomic unit across one or more streams, partitions, or state targets where supported. This capability borrows from Kafka transactions and relational commit semantics.

Acceptance criteria

- A transaction either commits all enclosed writes or makes none visible.
- Readers operating in committed mode never observe partial results from an unfinished transaction.
- Transaction metadata records begin, commit, abort, and timeout outcomes.
- Operational tooling can list open, long-running, aborted, and prepared transactions.
- Failure injection tests confirm atomicity across broker or node failure scenarios.

F-005 Log compaction and tombstone handling

TruthDB should support retention modes beyond simple deletion. For key-based streams it should compact obsolete versions and preserve the latest value or tombstone according to policy, similar to Kafka log compaction and state-store maintenance.

Acceptance criteria

- Streams can be configured for delete retention, compaction, or hybrid policies.
- Compaction preserves the latest record per key according to ordering rules.
- Tombstones are retained long enough to propagate deletion semantics safely to downstream consumers.
- Compaction progress, lag, and skipped segments are observable.
- Compaction never produces a state that violates the visible key history contract.

F-006 Tiered storage and cold replay

TruthDB should allow older segments to move to cheaper storage while remaining replayable. This preserves deep history without forcing all hot nodes to retain all bytes locally.

Acceptance criteria

- Retention policies can migrate segments between hot, warm, and cold tiers.
- Consumers can request replay from cold tiers without manual restore procedures.
- Cold replay exposes expected latency and throughput characteristics to clients.
- Tier transitions are policy driven and auditable.
- A node loss event does not permanently orphan cold data or metadata needed to fetch it.

F-007 Snapshotting for fast state reconstruction

TruthDB should be able to persist periodic snapshots for aggregates, materialized states, or projections so that replay starts from a checkpoint rather than from the beginning of time.

Acceptance criteria

- Snapshots can be created automatically by policy or explicitly by administrative action.
- A snapshot is versioned and tied to an exact source offset or event sequence.
- Recovery can start from the latest valid snapshot and continue replay from its checkpoint.
- If a snapshot is missing or corrupt, the system falls back to earlier snapshots or full replay.
- Snapshot format changes are versioned with clear upgrade rules.

F-008 Point-in-time recovery and rewind

TruthDB should support recovering storage and materialized views to a requested point in time or sequence boundary, combining database-grade recovery ideas with log-native replay.

Acceptance criteria

- An operator can specify a timestamp, offset, or transaction boundary as a recovery target.
- Recovery procedures clearly distinguish authoritative log state from derived state needing rebuild.
- Recovery metadata records the chosen target and the resulting consistent cut.
- The platform can validate that no records newer than the recovery target remain visible in recovered scopes.
- Runbooks and tooling support dry-run validation before destructive rewind operations.

2. Event sourcing, aggregates, and domain history

These features turn the raw log into a domain system. They capture intent, enforce aggregate boundaries, and preserve an explainable business narrative.

F-009 Aggregate streams and optimistic concurrency

TruthDB should support aggregate-centric streams where each entity or business object has a logical event history. Optimistic version checks prevent lost updates without requiring heavyweight locking.

Acceptance criteria

- Clients can append events with an expected aggregate version or last-seen sequence.
- A conflicting concurrent write is rejected with a deterministic version conflict result.
- The system returns the current authoritative version after a conflict.
- Aggregate reads can retrieve full history, latest snapshot, and latest materialized state.
- Concurrency behavior is documented and testable under parallel writers.

F-010 Domain event envelopes with metadata

Every event should carry a rich envelope, not just payload bytes. Metadata should include causation, correlation, actor, tenant, schema version, timestamps, and provenance needed for observability and replay reasoning.

Acceptance criteria

- Each event contains a standard metadata envelope alongside domain payload.
- The envelope includes unique event ID, correlation ID, causation ID, producer identity, and write timestamp at minimum.
- Metadata fields are queryable and indexable without decoding arbitrary payload formats.
- Envelope standards are enforced consistently across ingestion APIs.
- Audit exports preserve both payload and metadata without loss.

F-011 Temporal history and as-of views

TruthDB should make time a first-class concern. Users should be able to ask what the world looked like at a specific time, sequence, or valid-time interval. This borrows from temporal tables and event sourcing replay.

Acceptance criteria

- Queries can target current state, transaction-time history, and as-of snapshots.
- A record or entity can expose valid-from and valid-to semantics where appropriate.
- As-of queries return results consistent with a chosen time boundary, not mixed across later writes.
- Temporal APIs document timezone, clock source, and tie-breaking behavior.
- Retention policies define whether historical views are partial or complete.

F-012 Projection pipelines

TruthDB should support building read models, indexes, counters, search documents, and external feeds from the event log through managed projections. Projections are the bridge between immutable history and practical query shapes.

Acceptance criteria

- A projection can subscribe from a known offset and process events deterministically.
- Projection state records checkpoint, lag, health, and last error.
- Projection handlers are restart-safe and idempotent.
- Operators can replay, pause, resume, or rebuild a projection without data loss.
- A projection rebuild from history yields the same result as the continuously maintained version for deterministic handlers.

F-013 Dead-letter and poison-event strategy

TruthDB should define how projections and downstream handlers deal with malformed, unexpected, or repeatedly failing events. The platform needs a first-class dead-letter strategy rather than ad hoc operator intervention.

Acceptance criteria

- Processing failures can route events to a dead-letter stream or quarantine store with full context.
- Operators can inspect failure reason, original payload, retries, and handler version.
- Retry policies are configurable by error class and projection.
- Quarantined events can be reprocessed after remediation.
- Poison events do not silently stall unrelated processing pipelines.

F-014 Compensating actions and reversibility

Because immutable systems do not erase history, TruthDB should support compensating events, corrections, and supersession workflows. This is critical for real business processes where mistakes happen.

Acceptance criteria

- A correction is represented explicitly as a new event linked to the original event or business action.
- Read models can distinguish original, corrected, cancelled, and superseded facts.
- Audit views show the complete chain of change, not only the latest visible result.
- Business APIs can require justification metadata for corrective operations.
- Correction workflows do not break aggregate versioning or replay determinism.

F-015 Branching or sandbox replay

TruthDB should support spinning up isolated replay branches for testing, analytics, what-if simulation, or backfills. This allows safe experimentation without corrupting production truth.

Acceptance criteria

- A branch can be created from a selected offset, snapshot, or time boundary.
- Branch replay can use different projection code or schema mappings than production.
- Branch outputs remain isolated until explicitly promoted or discarded.
- Operators can compare branch results against baseline outputs.
- Branch creation preserves provenance back to the source dataset and cut point.

F-016 Event schema evolution

TruthDB should manage evolving event contracts without breaking old data. This includes versioned schemas, compatibility rules, and upcasting or translation when replaying historical events.

Acceptance criteria

- Each event schema has an explicit version and compatibility policy.
- The system can reject writes that violate registered schema rules.
- Replay of historical events supports upcasters or translation hooks where needed.
- Schema changes are auditable and tied to rollout metadata.
- Compatibility tests can validate whether a new handler version can read prior event versions.

3. Relational and transactional database capabilities

TruthDB should borrow the best ideas from PostgreSQL and SQL Server so it can behave like a serious database, not just an event pipe.

F-017 Declarative schema with typed columns and constraints

TruthDB should support declarative relational structures for cases where strong typing, constraints, and tabular contracts are the right model. Event-native storage and relational state should coexist.

Acceptance criteria

- Tables, views, and typed collections can be declared through DDL-like interfaces.
- Supported constraints include primary keys, unique constraints, foreign keys where applicable, check constraints, and not-null rules.
- Constraint violations return deterministic, machine-readable errors.
- Schema introspection APIs expose effective definitions and change history.
- Schema deployment can be validated before applying destructive changes.

F-018 MVCC or equivalent multi-version read isolation

TruthDB should provide snapshot-style reads so readers do not block writers and long-running reads can see a consistent view. PostgreSQL MVCC is the benchmark concept here.

Acceptance criteria

- A read transaction can observe a stable snapshot while concurrent writes continue.
- Read isolation levels and guarantees are explicitly documented.
- Garbage collection of old versions does not break active snapshots.
- Conflict and visibility rules are testable under concurrent read-write workloads.
- Operational metrics expose version churn and cleanup backlog.

F-019 Secondary indexes

TruthDB should support multiple index types over structured state and metadata. Indexes should accelerate point lookups, range scans, and filtered queries without changing the base truth model.

Acceptance criteria

- Users can create and drop indexes online or with controlled maintenance windows.
- The query planner can explain when and why an index is selected.
- Index builds expose status, errors, and resource usage.
- Index corruption or divergence is detectable and repairable.
- Write amplification introduced by indexes is observable and attributable.

F-020 Partitioned tables and state containers

TruthDB should support partitioning large relational or materialized datasets by key, time, tenant, or hybrid strategy. This improves scale, maintenance, and retention behavior.

Acceptance criteria

- Partitioned objects can be defined with explicit partition keys and policies.
- Queries can prune irrelevant partitions when filters make that possible.
- Retention, vacuum, compaction, and movement policies can operate per partition.
- Rebalancing or repartitioning procedures minimize downtime and preserve correctness.
- Metadata exposes skew, hot partitions, and partition growth trends.

F-021 JSON and semi-structured document support

TruthDB should not force every fact into rigid columns. It should support JSON or document-style attributes with indexing and query operators, similar to PostgreSQL JSONB and document search systems.

Acceptance criteria

- Semi-structured fields can be stored, filtered, and partially indexed.
- Users can query nested attributes without full payload scans when indexes exist.
- Schema-on-write and schema-flexible modes can coexist with clear validation rules.
- Document field extraction into derived columns or search documents is supported.
- Storage and query costs for large nested documents are observable.

F-022 Full-text search over structured state

TruthDB should support lexical full-text search over selected fields, complementing exact-match queries and event replay. This capability bridges database and search-engine usage.

Acceptance criteria

- Users can define searchable text fields and language analyzers where supported.
- Queries can return ranked matches, snippets, and relevance metadata.
- Search indexes update from transactions or event projections with bounded lag.
- Phrase, prefix, boolean, and field-scoped search are supported.
- Search behavior under schema or analyzer changes is versioned and testable.

F-023 Temporal tables for state history

For relationally modeled data, TruthDB should offer system-versioned history comparable to temporal tables. This provides history without forcing every consumer to rebuild from the raw event log.

Acceptance criteria

- A table can be configured to retain prior row versions automatically.
- Queries can address current rows and historical rows through explicit temporal syntax or API parameters.

- Historical retention policies are configurable by table or domain.
- Temporal history links clearly to originating transactions or events.
- Temporal reads are protected against accidental mixing of current and historical semantics.

F-024 Materialized views with controlled refresh

TruthDB should support managed materialized views for expensive derived queries. Refresh may be synchronous, incremental, event-driven, or scheduled depending on workload.

Acceptance criteria

- A materialized view declares source objects, refresh policy, and staleness contract.
- Clients can inspect last refresh time, source checkpoint, and lag.
- Incremental refresh is supported where source semantics allow it.
- Refresh failure handling is visible and recoverable.
- Query plans can distinguish live views from materialized views.

4. Search, analytics, and document retrieval

This section captures the Elasticsearch side of the vision: rich indexing, distributed search, aggregations, scoring, and lifecycle controls for large searchable datasets.

F-025 Distributed shards and replicas for search state

TruthDB search-oriented indexes should scale out through shards and replicas, enabling capacity growth, failover, and parallel query execution.

Acceptance criteria

- An index can be split into one or more primary shards with configurable replicas.
- Shard and replica placement are observable and policy controlled.
- A node failure can promote replica capacity without manual document reloads where replicas are current.
- Search requests can fan out across shards and merge results deterministically.
- Hot shards and allocation imbalance are detectable through operational metrics.

F-026 Mapping and analyzer management

TruthDB should support explicit field mappings and text analyzers for search indexes. This enables precise control over tokenization, normalization, relevance, and aggregation behavior.

Acceptance criteria

- Field types, analyzers, keyword subfields, and index options are declaratively defined.
- Mapping changes are versioned and validated before activation.
- Incompatible mapping changes require explicit migration or reindex paths.
- Analyzer behavior can be previewed and tested against sample text.
- Applications can inspect the active mapping and analyzer configuration programmatically.

F-027 Query DSL and composable filtering

TruthDB should offer an expressive query language that combines exact filters, full-text clauses, ranges, nested filters, and boolean composition.

Acceptance criteria

- Clients can express must, should, must-not, and filter-style clauses in one query object or syntax.
- Range, term, prefix, wildcard, phrase, and nested queries are supported where relevant.
- The engine can explain matching logic and relevance calculation.
- Query validation detects malformed combinations before execution where possible.
- Query execution statistics expose cost drivers such as scanned shards, skipped caches, and time spent per phase.

F-028 Aggregations and faceted analytics

TruthDB should support analytics over indexed data, not just document retrieval. Aggregations enable histograms, terms buckets, cardinality, percentiles, time-series rollups, and faceted navigation.

Acceptance criteria

- Queries can request aggregations alongside hits in one round trip.
- Bucket, metric, and pipeline-style aggregations are supported for declared field types.
- Aggregations can operate over filtered subsets without requiring separate preprocessing pipelines.
- Execution plans expose approximate versus exact modes where applicable.
- Large-cardinality aggregations surface memory and accuracy trade-offs to users.

F-029 Relevance ranking and tunable scoring

TruthDB should support configurable ranking behavior for search use cases. The system needs lexical relevance, boosts, tie-break rules, and optional business scoring overlays.

Acceptance criteria

- Search results return score metadata or an equivalent ranking explanation.
- Field boosts, recency boosts, and business-specific ranking functions are configurable.
- Users can opt for filter-only exact results when ranking is not desired.
- Ranking changes can be versioned and tested offline before rollout.
- Search explain APIs show why a result ranked where it did.

F-030 Reindexing and alias-based cutover

TruthDB should support safe reindexing when mappings, analyzers, schemas, or ranking rules change. Zero-downtime cutover patterns should be first class.

Acceptance criteria

- A new index generation can be built from existing truth sources without replacing the active index immediately.
- Clients can use logical aliases or named index handles instead of hard-coded physical index names.
- Cutover between generations is atomic from the client perspective.
- Rollback to the prior generation remains possible until explicitly retired.
- Reindex progress, throughput, and failures are observable.

F-031 Lifecycle management for search indexes

TruthDB should manage searchable data across its life cycle: rollover, shrink, retention, tiering, and deletion. This borrows from ILM concepts.

Acceptance criteria

- Policies can trigger rollover based on age, size, document count, or custom thresholds.

- Indexes can transition across performance or storage tiers under policy control.
- Deletion or archive actions require explicit safety conditions.
- Lifecycle state is visible per index generation.
- Policy simulation can predict lifecycle actions before activation.

F-032 Hybrid retrieval across structured, textual, and historical sources

TruthDB should unify retrieval across current relational state, search indexes, and historical event streams. This is one of the most distinctive hybrid opportunities.

Acceptance criteria

- A query can combine structured predicates, text search, and temporal boundaries in one request model.
- The response model makes clear which portions came from current state, search index, or historical replay.
- Cross-source consistency semantics are documented, including tolerated lag between sources.
- Fallback behavior is defined when one retrieval subsystem is degraded.
- Operators can measure latency contribution from each subsystem in hybrid queries.

5. Replication, synchronization, and streaming integration

These features make TruthDB a hub rather than an island. They cover CDC, subscriptions, replication, and external data movement.

F-033 Change data capture feeds

TruthDB should expose changes from relational or materialized state as ordered change feeds. This mirrors SQL Server CDC and log-based integration patterns.

Acceptance criteria

- Users can enable change capture for chosen objects with explicit retention and payload rules.
- Captured changes include operation type, key, before image where appropriate, after image, and commit ordering metadata.
- Consumers can resume from a stable checkpoint without rereading the entire base dataset.
- CDC capture does not require invasive application code changes.
- Operational tooling exposes capture lag, truncation horizon, and schema drift warnings.

F-034 Logical replication and subscriptions

TruthDB should support selective replication of data and events to other clusters, tenants, or regions. Replication should be logical and filterable, not only block- or file-based.

Acceptance criteria

- Publishers can define what objects, event types, or predicates are replicated.
- Subscribers track replication slots or durable checkpoints.
- Replication status includes lag, failures, and last applied sequence.
- Topology supports one-to-many and many-to-one patterns where conflict rules are defined.
- Replication can be resumed safely after outages without silent divergence.

F-035 Row or event filtering in replication pipelines

TruthDB should replicate only what is needed. Filters by tenant, event type, row predicate, or field selection reduce bandwidth and risk.

Acceptance criteria

- Replication rules support include and exclude filters at meaningful granularity.
- Filtering behavior is deterministic and testable with sample datasets.
- Operators can inspect why a given record was replicated or skipped.
- Schema evolution does not silently invalidate existing filter logic.
- Security rules prevent filters from leaking restricted data to unauthorized targets.

F-036 Outbox and external sink connectors

TruthDB should make it easy to publish committed changes to external buses, warehouses, search systems, caches, and applications through managed connector patterns.

Acceptance criteria

- A connector reads only committed changes and records durable progress.
- At-least-once or exactly-once guarantees are explicitly stated per sink type.
- Connector retries, backoff, and dead-letter behavior are configurable.
- Operators can pause, resume, and redeploy connectors without losing position.
- Connector health dashboards show lag, throughput, error rate, and last successful delivery.

F-037 Multi-region replication and disaster failover

TruthDB should support regional redundancy for high availability and disaster recovery. The product should distinguish local HA from inter-region DR because their trade-offs differ.

Acceptance criteria

- Replication modes state whether they are synchronous, semi-synchronous, or asynchronous.
- Recovery objectives for each topology are declared and measurable.
- Failover procedures are scriptable and auditable.
- After failover, clients can discover the new write leader or active endpoint automatically where configured.
- Split-brain protection and rejoin procedures are defined for network partition scenarios.

F-038 Backfill and bulk import with consistency controls

TruthDB should ingest historical datasets and large backfills without compromising ongoing live traffic. Bulk movement is a first-class operational need in real systems.

Acceptance criteria

- Bulk import jobs can write through validated pathways that preserve schema and ordering guarantees.
- Backfill can be tagged and isolated from live traffic for observability and throttling.
- The platform can define whether backfilled data should appear as historical events, current state upserts, or both.
- Duplicate and ordering reconciliation rules are explicit.
- A completed backfill emits audit metadata covering source, scope, time window, and counts.

6. Query model, API surface, and developer ergonomics

TruthDB should not only store truth; it should be pleasant and predictable to build on. This section reflects Redux, SQL, and modern data-platform usability ideas.

F-039 Unified API over commands, queries, and subscriptions

TruthDB should expose a coherent client model for writes, reads, and live updates. Developers should not have to learn unrelated protocols for each access pattern.

Acceptance criteria

- The client model distinguishes command operations, query operations, and subscription operations clearly.
- Authentication, authorization, tracing, and error shapes are consistent across operation types.
- SDKs can resume subscriptions and long-lived cursors after transient disconnects.
- The API surface documents ordering, visibility, and retry behavior for each operation class.
- Capability discovery allows clients to determine supported features at runtime.

F-040 Query planner introspection and explainability

TruthDB should explain how it will execute a query, projection, or retrieval plan. That is essential for performance work and for user trust.

Acceptance criteria

- Users can request an explain plan without executing the full workload where feasible.
- Plans identify chosen indexes, shard fan-out, estimated costs, and relevant filters.
- Execution statistics can be compared against estimated plans.
- The explain format is stable enough for automation and regression analysis.
- Security-sensitive details can be redacted in shared plans.

F-041 Saved queries, views, and reusable selectors

TruthDB should support named query definitions and reusable selectors, echoing both SQL views and Redux-style selector composition.

Acceptance criteria

- A saved query or selector can encapsulate common filters, joins, projections, or aggregations.
- Definitions are versioned and can be promoted across environments.
- Consumers can reference a saved query by stable name rather than embedding raw query text everywhere.
- Dependency tracking shows what applications or dashboards rely on a saved query.
- Breaking changes require compatibility review or explicit version increments.

F-042 Client-side cache coherence hints

TruthDB should help application state layers remain coherent. This is where Redux and RTK Query ideas become valuable: invalidation tags, freshness metadata, and subscription-driven updates.

Acceptance criteria

- Query responses can include freshness version, checkpoint, or invalidation metadata.
- Clients can subscribe to invalidation or update events for specific query scopes or entities.
- The API can surface whether a response is strongly current, checkpoint-consistent, or eventually consistent.
- SDK helpers can map server updates into normalized client caches.
- Cache invalidation rules are deterministic and testable for standard mutation patterns.

F-043 Normalized entity graph exposure

TruthDB should expose data in a way that supports normalized application state. Instead of only returning deeply nested blobs, it should optionally return entity collections and references that are easy to merge into client stores.

Acceptance criteria

- Query APIs can return entities plus reference edges or relationship metadata.
- The normalized response shape is stable and documented.
- Incremental updates can target individual entities without forcing full-object replacement.
- The system can include change reasons or source checkpoints alongside normalized entities.
- Developers can request denormalized or normalized forms explicitly.

F-044 Live query subscriptions

TruthDB should allow clients to subscribe to query result changes rather than polling. This blends streaming data with database queries and modern reactive UI expectations.

Acceptance criteria

- A client can open a live query and receive an initial consistent result plus subsequent changes.
- Changes include enough metadata to update a local result set deterministically.
- Subscriptions can resume from a checkpoint after reconnect where supported.
- Backpressure and slow-consumer behavior are defined.
- The platform measures subscription fan-out cost and per-subscriber lag.

F-045 Deterministic pagination and cursoring

TruthDB should support reliable pagination over mutable datasets and event streams. Offset pagination is often not enough; stable cursor semantics are required.

Acceptance criteria

- Queries can return opaque continuation tokens or cursors tied to a stable ordering.
- A cursor guarantees no duplicate or missing rows within the declared consistency window.
- The system documents how inserts and deletes affect pagination semantics.

- Expired or invalid cursors return clear errors and recovery guidance.
- Cursor performance and retention limits are observable.

F-046 DevTools-grade inspection and replay

TruthDB should offer a first-class inspection experience akin to Redux DevTools but for server-side state and history: browse changes, replay steps, diff states, inspect causation chains.

Acceptance criteria

- Operators and developers can inspect an entity or query result across a timeline of events or transactions.
- The tool can replay step by step and show before and after state deltas.
- Causation and correlation chains are navigable across services or event flows where metadata exists.
- Inspection views support filters by actor, tenant, event type, and time range.
- Permissions restrict access to sensitive payloads while preserving operational usefulness.

7. Security, tenancy, and governance

Serious data systems need more than raw power. They need principled boundaries, provenance, and governance. This section harvests security features seen in enterprise databases and modern platforms.

F-047 Authentication and workload identity

TruthDB should support strong service and user authentication, with clear identity propagation into audit metadata and policy decisions.

Acceptance criteria

- The platform supports pluggable authentication suitable for users, services, and automated jobs.
- Each request resolves to a stable authenticated identity before authorization occurs.
- Identity context is available to logs, traces, event metadata, and policy engines.
- Credential rotation and revocation are supported without cluster restarts.
- Failed authentication attempts are rate-limited and auditable.

F-048 Fine-grained authorization and row-level security

TruthDB should support policy-based access control down to row, event, field, tenant, or stream level, inspired by row-level security capabilities in major databases.

Acceptance criteria

- Policies can be defined at database, schema, object, row, event type, and field scope as applicable.
- The same policy model applies consistently across queries, live subscriptions, and export APIs.
- Policy decisions are explainable for debugging and audit.
- Unauthorized data is filtered or denied before it reaches the client or downstream sink.
- Replication and CDC paths respect the same effective access rules or clearly documented service-level overrides.

F-049 Tenant isolation and quota enforcement

TruthDB should safely host multiple tenants or domains with isolation guarantees around data, compute, and operational blast radius.

Acceptance criteria

- Tenants can be isolated logically and, where configured, physically.
- Quotas can limit storage, throughput, connections, search usage, and background job consumption.
- No tenant can read or infer another tenant's data through shared indexes, metadata, or error messages.
- Per-tenant metrics, throttling events, and policy violations are observable.

- Administrative actions can target a single tenant without destabilizing others.

F-050 Encryption, key management, and secret hygiene

TruthDB should protect data in transit and at rest, with operationally realistic key rotation and secret handling.

Acceptance criteria

- Transport encryption is enforced or configurable with safe defaults.
- At-rest encryption covers durable segments, snapshots, backups, and cold tiers where required.
- Key identifiers and rotation status are visible without exposing secret material.
- Rotating keys does not require destructive full-cluster rebuilds for supported modes.
- Secret access is minimized and auditable.

F-051 Audit trails and provenance graph

TruthDB should record who did what, when, from where, and under which policy or causation chain. Audit must cover both data and control-plane operations.

Acceptance criteria

- Administrative and data-plane operations are logged with actor, time, target, and outcome.
- Audit records are tamper-evident or protected by append-only controls.
- The system can link derived artifacts back to source events, schemas, and projection versions.
- Audit queries support investigation by actor, object, time range, and operation type.
- Retention and export rules for audit records are configurable and policy aware.

F-052 Data retention, legal hold, and deletion workflows

TruthDB should support nuanced retention governance. Some data must expire, some must be held, and some must be deleted in a traceable manner despite immutable internals.

Acceptance criteria

- Retention rules can be defined by object type, tenant, stream, index, or legal category.
- Legal hold can suspend deletion or compaction actions for protected scopes.
- Deletion workflows record rationale, initiator, and affected identifiers.
- The system distinguishes logical erasure, physical purge eligibility, and historical audit preservation.
- Operators can simulate retention actions before execution.

8. Operations, observability, and performance management

This section captures the features that make TruthDB operable in production: metrics, balancing, background maintenance, tuning, and failure visibility.

F-053 Built-in observability for writes, reads, and lag

TruthDB should expose first-class metrics, traces, and logs for every critical subsystem. Operators must be able to see ingestion health, query latency, projection lag, replica state, and storage pressure.

Acceptance criteria

- The system emits structured metrics for throughput, latency, error rate, lag, storage, and background maintenance.
- Tracing can follow a command from ingress to commit, projection, and external sink where integrated.
- Dashboards can break down performance by tenant, stream, query class, and node.
- Alert conditions for common failure modes are documented and measurable.
- Metrics retention and cardinality controls are available so observability remains sustainable.

F-054 Resource governance and workload classes

TruthDB should isolate competing workloads such as transactional writes, replay, search, CDC export, and ad hoc analytics. Without this, one success mode will destroy another.

Acceptance criteria

- Workloads can be assigned to classes with quotas or scheduler priorities.
- Heavy replay or reindex jobs can be throttled independently of foreground traffic.
- Operators can inspect queue depth and resource consumption by workload class.
- Admission control rejects or defers work when service-level policies require it.
- Performance isolation tests confirm that one noisy workload cannot starve critical writes under configured policies.

F-055 Automatic maintenance: vacuum, compaction, checkpointing, and cleanup

TruthDB should automate low-level maintenance analogous to PostgreSQL vacuum, log cleanup, and search segment maintenance, while still exposing enough control for experts.

Acceptance criteria

- Background maintenance tasks run automatically according to safe defaults and configurable policies.
- Operators can inspect maintenance backlog, last run, duration, and impact.
- Maintenance tasks can be paused or tuned during incidents or migrations.
- The system warns before maintenance debt threatens correctness or capacity.

- Automatic maintenance never silently changes user-visible semantics beyond documented retention and cleanup behavior.

F-056 Cluster rebalancing and hot-spot mitigation

TruthDB should identify and relieve hotspots across partitions, shards, indexes, and tenants. Balance is not optional in a hybrid system.

Acceptance criteria

- The platform detects skew in storage, throughput, query load, and replication load.
- Rebalancing can move partitions or shards with visible progress and safety checks.
- Hotspot diagnostics identify dominant keys, queries, tenants, or pipelines.
- Operators can apply throttling, isolation, or repartition suggestions based on observed skew.
- Rebalancing procedures document temporary effects on latency, ordering, or redundancy where relevant.

F-057 Backup, restore, and verification

TruthDB should support backups across log, metadata, snapshots, and derived state with verification that they are actually restorable.

Acceptance criteria

- Backups cover all required components to reconstruct an operational system, including metadata needed to interpret data.
- Restore procedures can validate consistency before activation.
- Scheduled backup verification tests are supported and report success or failure.
- Backup catalogs record scope, timestamp, encryption status, and retention class.
- Restore can target alternate environments for drill testing and investigation.

F-058 Upgrade, migration, and compatibility controls

TruthDB should provide disciplined upgrade paths for nodes, clients, schemas, and storage formats. A system this ambitious will live through many generations.

Acceptance criteria

- Version compatibility matrices define supported mixed-version states during rolling upgrades.
- Storage or protocol format upgrades are explicit and reversible where feasible.
- Upgrade preflight checks identify blockers such as deprecated mappings, unhealthy replicas, or incompatible schemas.
- Operators can stage upgrades in canary scope before full rollout.
- Post-upgrade verification confirms cluster health, data consistency, and feature activation state.

9. Advanced differentiators worth considering

These are not mandatory for a first release, but they are the kinds of features that could make TruthDB meaningfully better than copying any single existing product.

F-059 Unified consistency contracts per query

TruthDB should let a client ask explicitly for the consistency envelope it wants: latest-committed, partition-stable, globally checkpointed, temporal as-of, or eventually indexed search.

Acceptance criteria

- Each query declares or inherits a consistency mode.
- Responses report the effective consistency level and relevant checkpoint metadata.
- The platform rejects unsupported combinations clearly instead of silently degrading guarantees.
- Monitoring can track how much traffic uses each consistency contract.
- Documentation explains latency and availability trade-offs for each mode.

F-060 Cross-model lineage between event, row, document, and cache

TruthDB should be able to show how one business fact propagated through multiple representations: raw event, aggregate state, relational row, search document, cache invalidation, and external export.

Acceptance criteria

- Each derived artifact can reference its upstream source event or transaction checkpoints.
- Lineage queries can traverse from source to derived outputs and back.
- Lineage metadata survives reindexing, view refresh, and replay where practical.
- Operators can identify stale derivatives that lag behind their source truth.
- Lineage access respects security policies on all traversed artifacts.

F-061 Deterministic historical recomputation service

TruthDB should provide a managed facility to recompute any derived dataset under a selected code version, schema version, and replay boundary. This would operationalize one of the biggest promises of event sourcing.

Acceptance criteria

- A recomputation job records source range, code version, schema version, and target artifact.
- The result is reproducible when rerun with the same declared inputs.
- Differences from the incumbent artifact can be diffed before promotion.
- Resource use and execution time are observable and controllable.
- Promotion from recomputed artifact to live artifact uses an atomic or clearly staged procedure.

F-062 Policy-driven storage model selection

TruthDB should allow the platform or developer to declare the best representation for each domain: append-only event stream, relational state, document index, analytical projection, or hybrid. The system should make these trade-offs explicit rather than accidental.

Acceptance criteria

- A dataset definition can specify its authoritative model and any derived models.
- The platform can validate that required invariants are compatible with the chosen model.
- Migration between models is supported through explicit workflows, not hidden implementation changes.
- Introspection shows why a dataset is stored and indexed the way it is.
- Operational tooling can estimate cost and performance implications of an alternate model choice.

Recommended next step after this catalogue

Turn this document into a layered roadmap. A practical next artifact would group these features into: (1) foundational storage and protocol primitives, (2) first useful product slice, (3) search and projection expansion, (4) governance and enterprise hardening, and (5) long-horizon differentiators. That next document should also mark which features are authoritative-system features versus derived or optional subsystems.

Reference sources used for synthesis

1. Apache Kafka official documentation: core concepts, producer idempotence, transactions, and exactly-once processing semantics.
2. Elastic official documentation: Elasticsearch shards and replicas, mappings, aggregations, data tiers, and index lifecycle management.
3. PostgreSQL official documentation and feature matrix: MVCC, autovacuum, partitioning, JSON capabilities, full-text search, and logical replication.
4. Microsoft Learn for SQL Server and Azure SQL: temporal tables, row-level security, change data capture, columnstore, in-memory OLTP, and recent platform features.
5. Redux official documentation: Redux Toolkit, normalized state, memoized selectors, listener middleware, RTK Query, and DevTools patterns.
6. Azure Architecture Center: event sourcing and CQRS patterns for append-only stores, projections, and replay-driven system design.